

Data Structures (CS2001)

Course Instructor(s):

Ms. Umarah Qaseem, Ms. Kainat Iqbal

Section(s): AI (A,B)

Sessional-II Exam

Total Time (Hrs): 1

Total Marks: 45

Total Questions: 4

Date: Apr 15, 2026

Student Signature

Roll No

Course Section

Do not write below this line.

Attempt all the questions.

Instructions:

1. Comprehension is part of the exam. Read the questions carefully.
2. Write ONLY in the given space provided. Writing outside the boxes will not be considered.

	Q1	Q2	Q3	Q4	Total
Total Marks	5	5	20	15	45
Marks Obtained	01	01	06	01	

[CLO 4]

Question 01: Determine the exact output produced by the given C++ function when it is executed on the Binary Search Tree (BST) shown below. [5 Marks]

- Write the output level by level.
- Preserve the exact order of elements as printed by the function.
- Each level must appear on a new line, exactly as in the program output.

Code	Output
<pre> 1 void treeTraversal(Node* root) { 2 if (!root) return; 3 queue<Node*> q; 4 q.push(root); 5 bool flag = true; 6 while (!q.empty()) { 7 int size = q.size(); 8 int* arr = new int[size]; 9 for (int i = 0; i < size; i++) { 10 Node* node = q.front(); 11 q.pop(); 12 int index = flag ? i : (size-1-i); 13 arr[index] = node->data; 14 if (node->left) 15 q.push(node->left); 16 if (node->right) 17 q.push(node->right); 18 } 19 for (int i = 0; i < size; i++) 20 cout << arr[i] << " "; 21 cout << endl; 22 delete[] arr; 23 flag = !flag; 24 } 25 } 26 </pre>	BST <pre> graph TD 56 --> 30 56 --> 70 30 --> 22 30 --> 40 22 --> 11 11 --> 3 11 --> 16 70 --> 60 70 --> 95 60 --> 65 65 --> 63 65 --> 67 </pre> Output of the code: <pre> 56 30 70 22 40 60 95 11 65 3 16 63 67 </pre>

National University of Computer and Emerging Sciences

Islamabad Campus

[CLO 2]

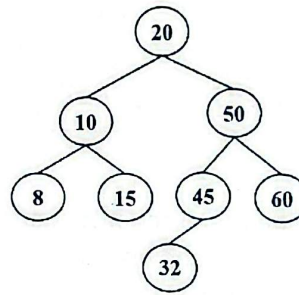
Question 02: Write the time complexity in the form of $O()$ for the following operations. [5 marks]

Sr. no	Operation	Time Complexity
1	Reversing a queue using one additional stack	$O(n)$ ✓
2	Converting a Circular Doubly Linked List into a Doubly Linked List	$O(n^2)$ ✗
3	Deleting a node given a pointer to that node in a Doubly Linked List	$O(n)$ ✗
4	Printing all root-to-leaf paths in a binary tree with n nodes and height h	$O(\log n)$ ✗
5	Finding the minimum element in AVL	$O(n)$ ✗

[CLO 1: Demonstrate basic concepts of data structure and algorithms.]

Question 03: Consider the following AVL tree: Perform the following operations sequentially on the given AVL tree: [20 Marks]

- Insert (40)
- Insert (35)
- Insert (30)
- Insert (31)
- Insert (5)
- Delete (35)
- Delete (50)



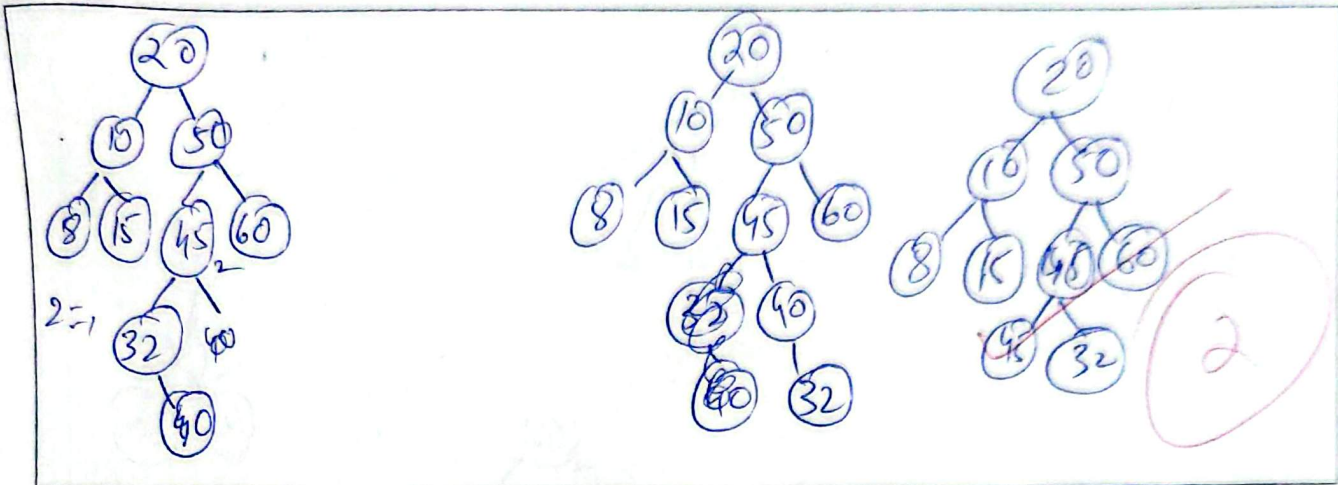
For each operation, you must:

1. Update the balance factor of all affected nodes.
2. Identify the first node that becomes unbalanced (if any).
3. Specify the imbalance case:
 - LL (Left-Left) imbalance
 - RR (Right-Right) imbalance
 - LR (Left-Right) imbalance
 - RL (Right-Left) imbalance
4. Perform the required rotation(s) to restore AVL balance.
5. Draw the resulting AVL tree after each operation.

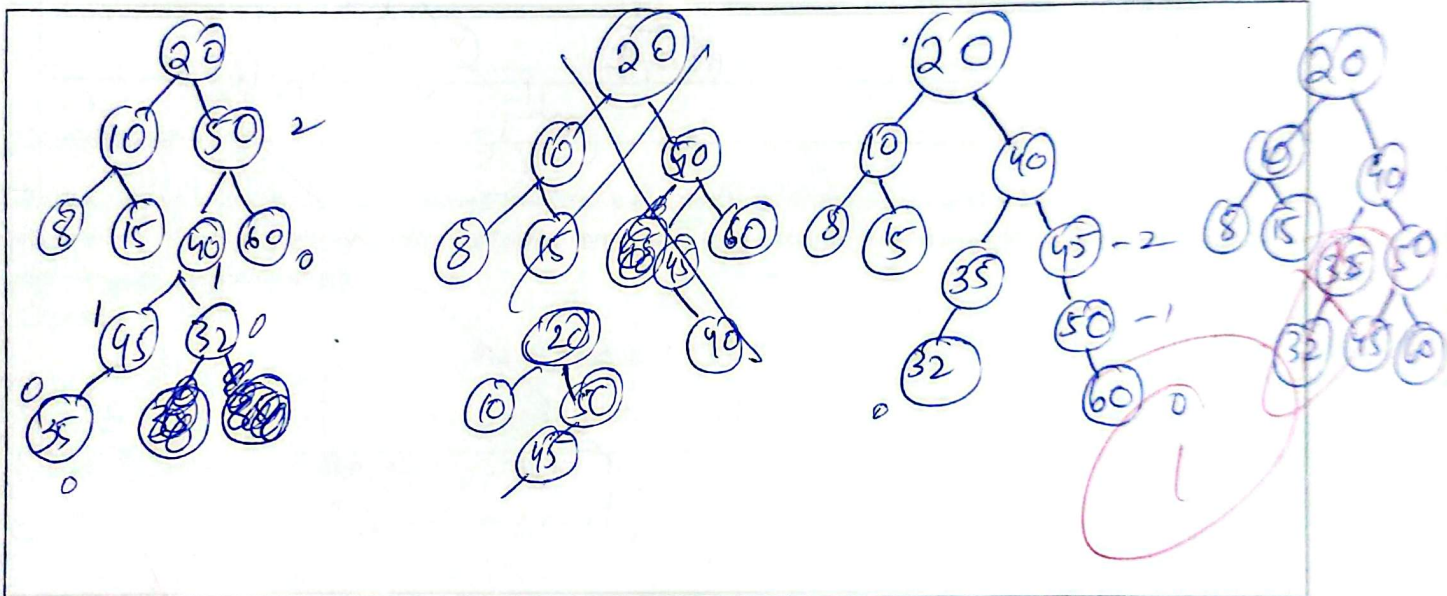
Fill the given table after each operation.

Part No.	Operation	First Unbalanced Node (if any)	Balance Factor of that Node	Imbalance Case	Rotation Performed	Root After Operation	Height of Tree
1	Insert (40)	45	2	RL	RL/RR	20	3
2	Insert (35)	50	2	LL	LL/RR	20	3
3	Insert (30)	35	2	LL	LL	20	3
4	Insert (31)	20	-2	RLR	LR/L		
5	Insert (5)						
6	Delete (35)						
7	Delete (50)						

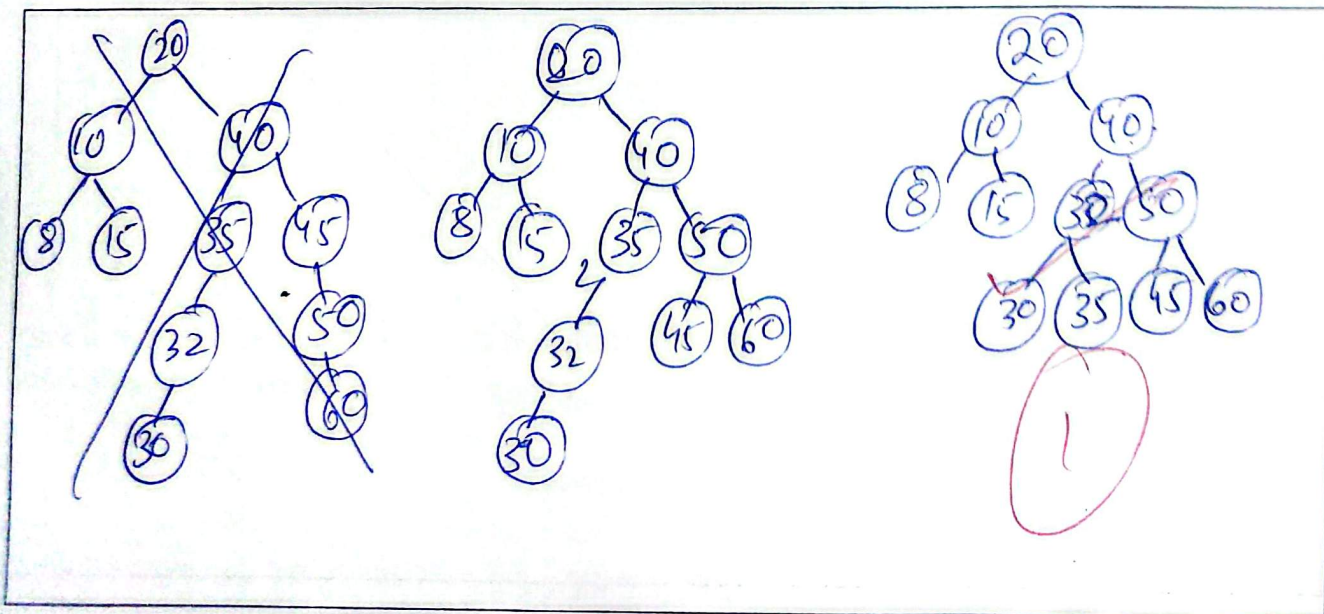
1. Insert (40)



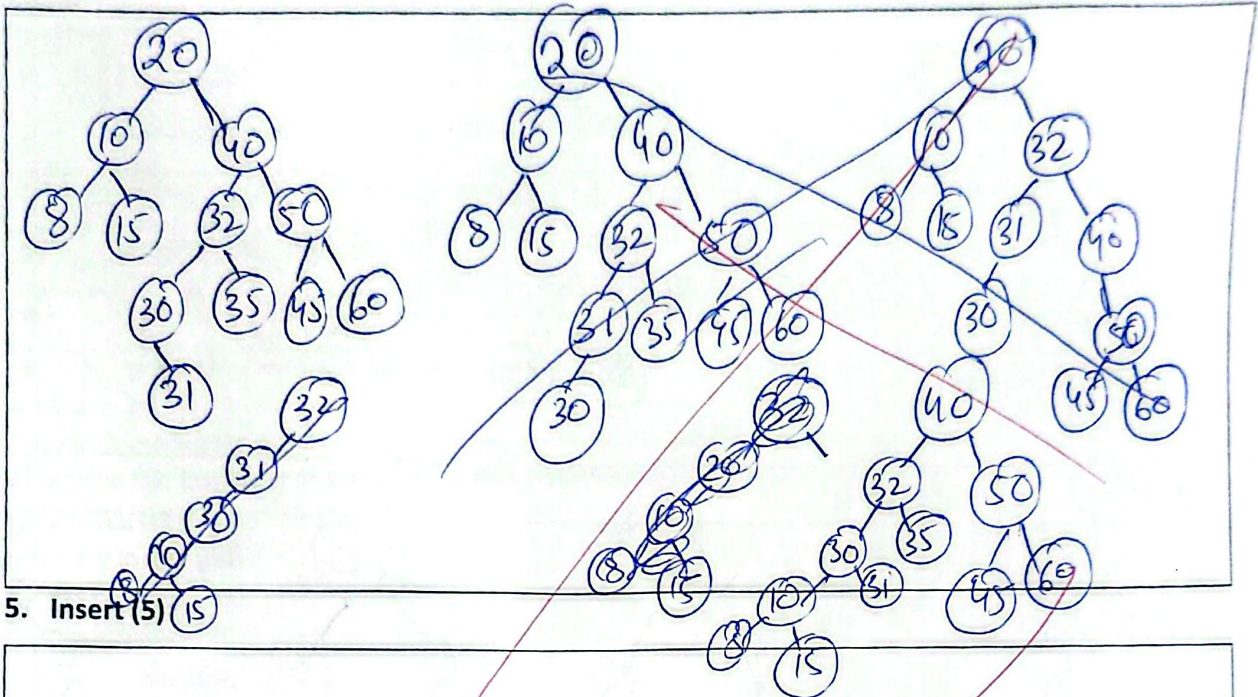
2. Insert (35)



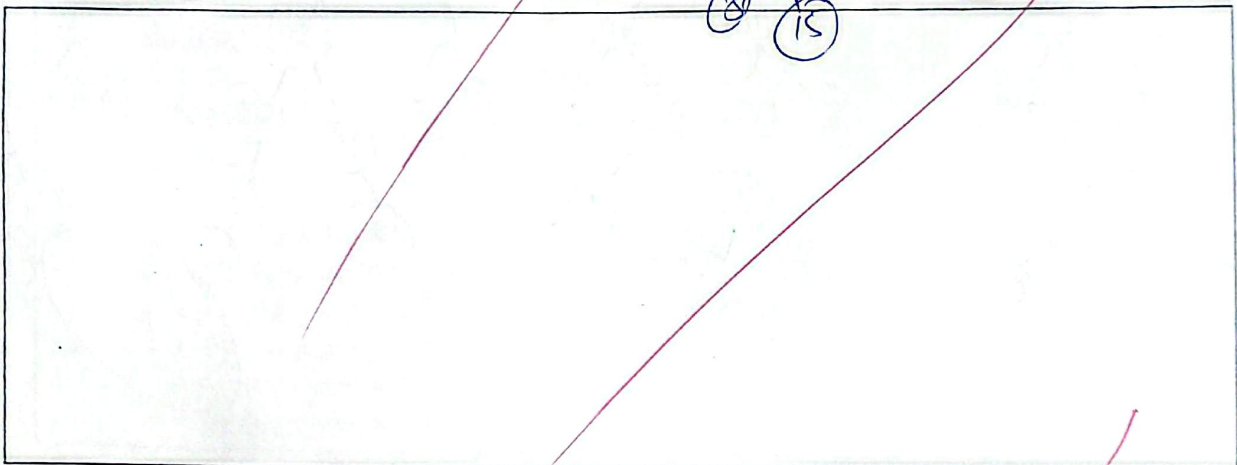
3. Insert (30)



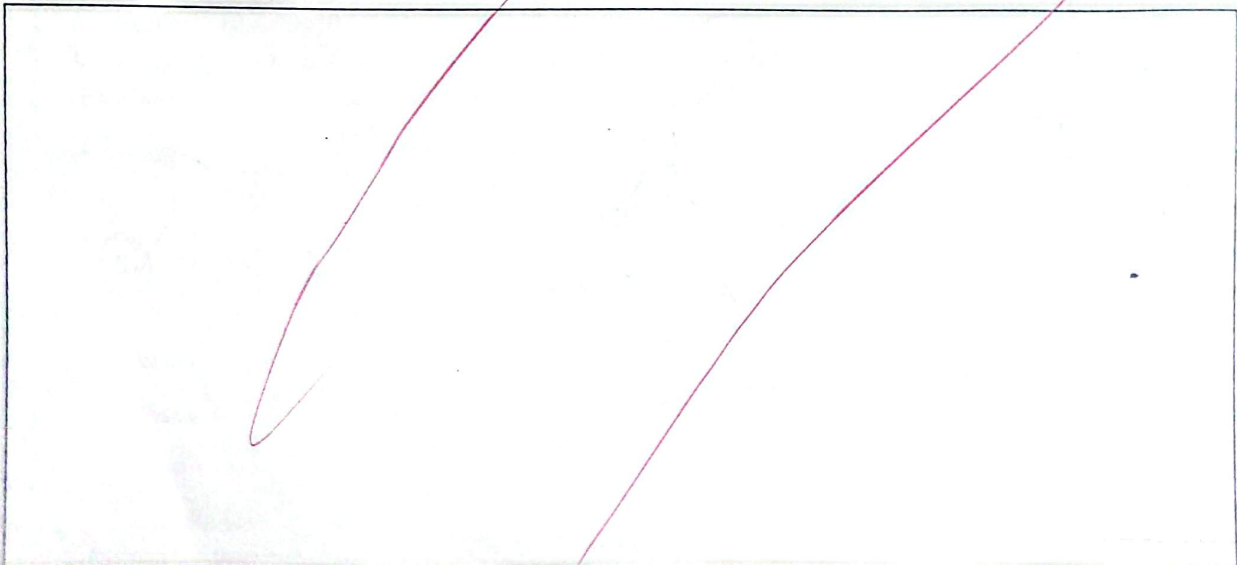
4. Insert (31)



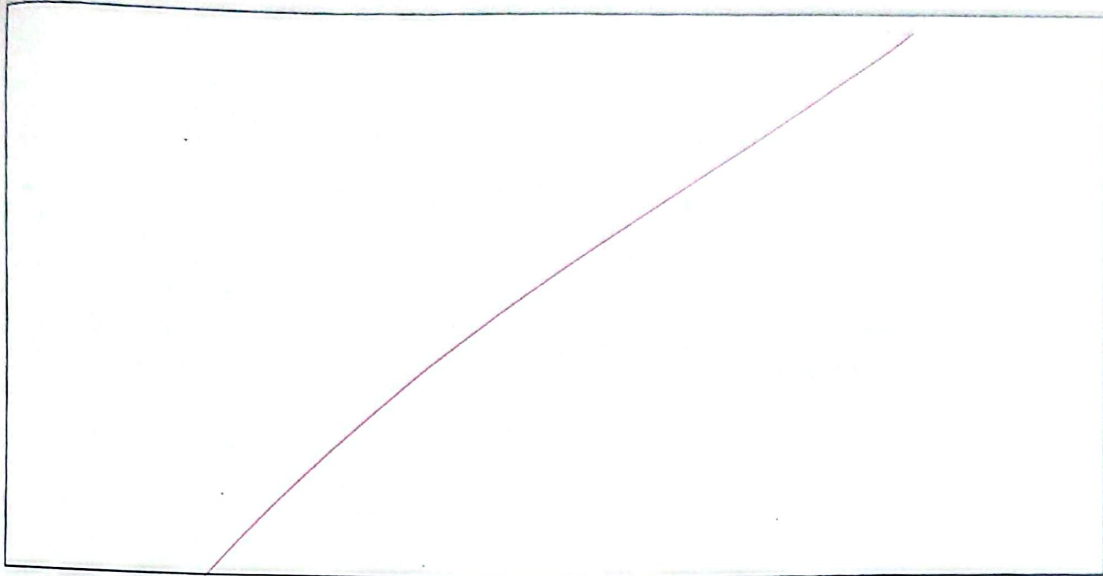
5. Insert (5)



6. Delete (35)



7. Delete (50) using inorder successor method



[CLO 3]

Question 04:

[15 Marks]

Part A: Write a C++ function to move the first k elements of the queue to the back, while preserving their order. *Do not use any additional data structures and assume that all necessary Queue functions are already implemented.*

[5 Marks]

Example:

Input: Queue: 1 2 3 4 5, k = 2

Output: 3 4 5 1 2



```
void rotateQueue(Queue& q, int k) {  
    node * temp = head;
```

```
}
```

Part B: You are given a Binary Search Tree (BST). Write a C++ function to convert the given BST into a sorted Doubly Linked List (DLL) in-place.

[10 Marks]

- Use the same node structure:
 - left pointer should act as **previous**
 - right pointer should act as **next**
- You are required to complete the function using the following prototype.

Note: Assume that all necessary functions are already implemented.

```
struct Node {  
    int data;  
    Node* left, * right;  
    Node(int val) : data(val), left(nullptr), right(nullptr) {}  
};  
void convertToDLL(Node* root, Node*& head, Node*& prev) {
```

*node * temp = root
while(temp->value <*

```
};
```

Bonus Question: A circular linked list has N nodes. You start at head and take exactly $2N + 3$ steps forward. Which node are you on? 5 [2 Marks]

$2(N) + 3$

$2 + 3$

Good Luck!